Guida di riferimento ai contratti

Bozza di struttura — campione: librerie mem e math, due moduli ciascuna

Indice

1 Libreria mem **3**
1.1 mem16.asm 5
1.1.3 m_Mov16 7
1.1.4 m_Swap16 9
1.1.5 m_CmpU16 11
1.2 mem24.asm 13
1.2.3 m_Mov24 15
1.2.4 m_Swap24 17
1.2.5 m_CmpU24 19
2 Libreria math **21**
2.1 math_mult.asm 23
2.1.2 Note d'uso 24
2.1.3 m_Mul_Negate 27
2.1.4 m_Mul_Stage 29
2.1.5 m_Mul_Core 31
2.2 math_div.asm 33
2.2.2 Note d'uso 34
2.2.3 m_Div_Negate 37
2.2.4 m_Div_Roll 39
2.2.5 m_Div_Core_Byte 41

Libreria

mem

1 Libreria mem

mem16.asm Macro per la manipolazione di valori a 16 bit.

mem24.asm Macro per la manipolazione di valori a 24 bit.

1.1 mem16.asm

1.1.1 Descrizione

Macro per la manipolazione di valori a 16 bit.

```
m_Mov16   Sposta un valore o un puntatore a 16 bit
m_Swap16  Scambia due puntatori a 16 bit
m_CmpU16  Confronta due valori a 16 bit senza segno
m_CmpS16  Confronta due valori a 16 bit con segno
m_Inc16   Incrementa un contatore a 16 bit
m_Dec16   Decrementa un contatore a 16 bit
```

1.1.2 Note d'uso

Nessuna nota in questo modulo.

1.1.3 m_Mov16

```
; ================================================================================
; MACRO: m_Mov16
; --------------------------------------------------------------------------------
; Descrizione:  Copia un valore costante o un puntatore a 16 bit dalla sorgente
;               alla destinazione, secondo l'indirizzamento scelto.
;
;                 "IMM"  Immediato: .src_ è un valore a 16 bit e .dest_ è un
;                        indirizzo o una coppia di registri.
;                 "ABS"  Assoluto: .src_ e .dest_ sono indirizzi assoluti; uno
;                        dei due può essere una coppia di registri.
;                 "ZP"   Pagina Zero: .src_ e .dest_ sono indirizzi in pagina
;                        zero; uno dei due può essere una coppia di registri.
;
;               Le coppie di registri si scrivono ".<hi><lo>" (byte alto e byte
;               basso): .AX, .AY, .XY, .XA, .YA e .YX.
;
;               Con "IMM" una costante con i due byte uguali (per esempio 0,
;               $FFFF o $4242) usa una sequenza più breve. Con "ABS" e "ZP", da
;               indirizzo a indirizzo, la copia procede nel verso che non altera
;               il sorgente se le due aree si sovrappongono.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .src_ (2 byte, Macro)   : Valore (IMM), indirizzo o coppia di registri
;   - .dest_ (2 byte, Macro)  : Indirizzo o coppia di registri
;
; Output:
;   - .dest_ (2 byte, Macro)  : Riceve il valore di .src_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.4 m_Swap16

```
; ================================================================================
; MACRO: m_Swap16
; --------------------------------------------------------------------------------
; Descrizione:  Scambia il valore di due puntatori a 16 bit usando i registri.
;
; Input:
;   - .addr1_ (2 byte, Macro) : Indirizzo del primo valore
;   - .addr2_ (2 byte, Macro) : Indirizzo del secondo valore
;
; Output:
;   - .addr1_ (2 byte, Macro) : Riceve il valore di .addr2_
;   - .addr2_ (2 byte, Macro) : Riceve il valore di .addr1_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   <system/standard.asm>
; ================================================================================
```

1.1.5 m_CmpU16

```
; ================================================================================
; MACRO: m_CmpU16
; --------------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 16 bit senza segno, secondo
;               l'indirizzamento scelto. Il risultato è nei flag, come con
;               l'istruzione CMP.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo o una coppia di
;                        registri, .op2_ è un valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti; .op1_
;                        può essere una coppia di registri.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero; .op1_ può essere una coppia di registri.
;
;               Le coppie di registri si scrivono ".<hi><lo>" (byte alto e byte
;               basso): .AX, .AY, .XY, .XA, .YA e .YX. Con una coppia di
;               registri come .op1_ nessun registro viene alterato.
;
;               Con "IMM" e .op1_ indirizzo, i confronti con 0 e con $FFFF usano
;               una sequenza più breve.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (2 byte, Macro)   : Primo operando: indirizzo o coppia di registri
;   - .op2_ (2 byte, Macro)   : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Zero (1 bit, .P)   : 0 = .op1_ diverso da .op2_; 1 = uguali
;   - Flag Carry (1 bit, .P)  : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                               segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.2 mem24.asm

1.2.1 Descrizione

Macro per la manipolazione di valori a 24 bit.

```
m_Mov24   Sposta un valore o un puntatore a 24 bit
m_Swap24  Scambia due puntatori a 24 bit
m_CmpU24  Confronta due valori a 24 bit senza segno
m_CmpS24  Confronta due valori a 24 bit con segno
m_Inc24   Incrementa un contatore a 24 bit
m_Dec24   Decrementa un contatore a 24 bit
```

1.2.2 Note d'uso

Nessuna nota in questo modulo.

1.2.3 m_Mov24

```
; ================================================================================
; MACRO: m_Mov24
; --------------------------------------------------------------------------------
; Descrizione:  Copia un valore costante o un puntatore a 24 bit dalla sorgente
;               alla destinazione, secondo l'indirizzamento scelto.
;
;                 "IMM"  Immediato: .src_ è un valore a 24 bit e .dest_ è un
;                        indirizzo o una terna di registri.
;                 "ABS"  Assoluto: .src_ e .dest_ sono indirizzi assoluti; uno
;                        dei due può essere una terna di registri.
;                 "ZP"   Pagina Zero: .src_ e .dest_ sono indirizzi in pagina
;                        zero; uno dei due può essere una terna di registri.
;
;               Le terne di registri si scrivono ".<hi><mid><lo>" (byte alto,
;               intermedio e basso): .AXY e .YXA.
;
;               Con "IMM" una costante con i tre byte uguali (per esempio 0,
;               $FFFFFF o $424242) usa una sequenza più breve. Con "ABS" e "ZP",
;               da indirizzo a indirizzo, la copia procede nel verso che non
;               altera il sorgente se le due aree si sovrappongono.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .src_ (3 byte, Macro)   : Valore (IMM), indirizzo o terna di registri
;   - .dest_ (3 byte, Macro)  : Indirizzo o terna di registri
;
; Output:
;   - .dest_ (3 byte, Macro)  : Riceve il valore di .src_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.2.4 m_Swap24

```
; ================================================================================
; MACRO: m_Swap24
; --------------------------------------------------------------------------------
; Descrizione:  Scambia il valore di due puntatori a 24 bit usando i registri.
;
; Input:
;   - .addr1_ (3 byte, Macro) : Indirizzo del primo valore
;   - .addr2_ (3 byte, Macro) : Indirizzo del secondo valore
;
; Output:
;   - .addr1_ (3 byte, Macro) : Riceve il valore di .addr2_
;   - .addr2_ (3 byte, Macro) : Riceve il valore di .addr1_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   <system/standard.asm>
; ================================================================================
```

1.2.5 m_CmpU24

```
; ================================================================================
; MACRO: m_CmpU24
; --------------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 24 bit senza segno, secondo
;               l'indirizzamento scelto. Il risultato è nei flag, come con
;               l'istruzione CMP.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo o una terna di
;                        registri, .op2_ è un valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti; .op1_
;                        può essere una terna di registri.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero; .op1_ può essere una terna di registri.
;
;               Le terne di registri si scrivono ".<hi><mid><lo>" (byte alto,
;               intermedio e basso): .AXY e .YXA. Con una terna di registri come
;               .op1_ nessun registro viene alterato.
;
;               Con "IMM" e .op1_ indirizzo, i confronti con 0 e con $FFFFFF
;               usano una sequenza più breve.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (3 byte, Macro)   : Primo operando: indirizzo o terna di registri
;   - .op2_ (3 byte, Macro)   : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Zero (1 bit, .P)   : 0 = .op1_ diverso da .op2_; 1 = uguali
;   - Flag Carry (1 bit, .P)  : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                               segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

Libreria

math

2 Libreria math

math_mult.asm Moltiplicazione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il prodotto ha sempre la lunghezza esatta m+n bit, senza perdita di precisione.

math_div.asm Divisione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il quoziente ha la lunghezza del dividendo e il resto quella del divisore, senza perdita di precisione.

2.1 math_mult.asm

2.1.1 Descrizione

Moltiplicazione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il prodotto ha sempre la lunghezza esatta m+n bit, senza perdita di precisione.

```
m_Mul_Negate      Nega un valore a più byte in pagina zero
m_Mul_Stage       Elabora un byte del moltiplicatore
m_Mul_Core        Nucleo generico senza segno
m_Mul_Core_8x8    Nucleo senza segno 8x8 bit
m_Mul_Core_16x8   Nucleo senza segno 16x8 bit
m_Mul_Core_24x8   Nucleo senza segno 24x8 bit
m_Mul_Core_32x8   Nucleo senza segno 32x8 bit
m_Mul_Core_16x16  Nucleo senza segno 16x16 bit
m_Mul_Core_24x16  Nucleo senza segno 24x16 bit
m_Mul_Core_32x16  Nucleo senza segno 32x16 bit
m_Mul_Core_24x24  Nucleo senza segno 24x24 bit
m_Mul_Core_32x24  Nucleo senza segno 32x24 bit
m_Mul_Core_32x32  Nucleo senza segno 32x32 bit
MUL_8X8           Prodotto 8x8 bit senza segno
SMUL_8X8          Prodotto 8x8 bit con segno
MUL_16X8          Prodotto 16x8 bit senza segno
SMUL_16X8         Prodotto 16x8 bit con segno
MUL_24X8          Prodotto 24x8 bit senza segno
SMUL_24X8         Prodotto 24x8 bit con segno
MUL_32X8          Prodotto 32x8 bit senza segno
SMUL_32X8         Prodotto 32x8 bit con segno
MUL_16X16         Prodotto 16x16 bit senza segno
SMUL_16X16        Prodotto 16x16 bit con segno
MUL_24X16         Prodotto 24x16 bit senza segno
SMUL_24X16        Prodotto 24x16 bit con segno
MUL_32X16         Prodotto 32x16 bit senza segno
SMUL_32X16        Prodotto 32x16 bit con segno
MUL_24X24         Prodotto 24x24 bit senza segno
SMUL_24X24        Prodotto 24x24 bit con segno
MUL_32X24         Prodotto 32x24 bit senza segno
SMUL_32X24        Prodotto 32x24 bit con segno
MUL_32X32         Prodotto 32x32 bit senza segno
SMUL_32X32        Prodotto 32x32 bit con segno
```

2.1.2 Note d'uso

Vedi le pagine seguenti.

2.1.2 Note d'uso

```
; NOTA #01
; --------
; Il primo operando è il più lungo e fa da moltiplicando; il secondo è il
; moltiplicatore. Le routine con operandi in registri usano, dal byte più
; basso al più alto, .A, .X e .Y (cioè la terna .YXA, come nell'ordine di
; memoria). I byte del prodotto oltre il terzo non entrano nei registri: si
; leggono da MUL_RES+3 in poi.
;
;   Routine  1º operando    2º operando    Registri in uscita
;   -------  -------------  -------------  ------------------
;   8x8      .A             .X             .A, .X
;   16x8     .A basso,      .Y             .A, .X, .Y
;            .X alto
;   24x8     MUL_1 (ZP)     .A             .A, .X, .Y
;   32x8     MUL_1 (ZP)     .A             .A, .X, .Y
;   16x16    MUL_1 (ZP)     .A basso,      .A, .X, .Y
;                           .X alto
;   24x16    MUL_1 (ZP)     .A basso,      .A, .X, .Y
;                           .X alto
;   32x16    MUL_1 (ZP)     .A basso,      .A, .X, .Y
;                           .X alto
;   24x24    MUL_1 (ZP)     .A, .X, .Y     .A, .X, .Y
;   32x24    MUL_1 (ZP)     .A, .X, .Y     .A, .X, .Y
;   32x32    MUL_1 (ZP)     MUL_2 (ZP)     .A, .X, .Y
;
; Per le versioni con segno gli operandi sono in complemento a 2 e il prodotto
; ha il segno corretto. L'input non si conserva: il moltiplicatore viene
; consumato e, nelle versioni con segno, gli operandi diventano positivi.
```

```
; NOTA #02
; --------
; Le routine lavorano in pagina zero sui 12 byte da NUMWRK a NUMWRK+11, cioè
; NUMWRK più i primi due byte di FAC, che sono liberi finché non si usa la
; matematica in virgola mobile del BASIC.
;
;   MUL_1    NUMWRK+0 .. NUMWRK+3     Moltiplicando (primo operando)
;   MUL_2    NUMWRK+4 .. NUMWRK+7     Moltiplicatore (secondo operando)
;   MUL_RES  NUMWRK+4 .. NUMWRK+11    Prodotto, da 2 a 8 byte
;
; MUL_2 e MUL_RES coincidono: il moltiplicatore occupa i byte bassi del
; prodotto e viene consumato mentre i byte definitivi prendono il suo posto,
; bit dopo bit. I byte alti di MUL_RES ospitano l'accumulatore. Per questo il
; moltiplicatore non si conserva e MUL_2 e MUL_RES non si usano insieme.
;
; La stessa area è condivisa con ZPL_1 e ZPL_2 (puntatori utente a 24/32 bit)
; e con le routine di divisione: non si possono usare contemporaneamente.
```

2.1.2 Note d'uso (continua)

```
; NOTA #03
; --------
; Si elabora un byte del moltiplicatore alla volta, con 8 iterazioni di
; shift-and-add: se il bit meno significativo vale 1 si somma il moltiplicando
; all'accumulatore alto, poi tutto scorre di un bit a destra. Il moltiplicando
; non si sposta mai e ogni iterazione fa scorrere solo l'accumulatore e il
; byte in esame del moltiplicatore, non l'intero prodotto. Il byte più alto
; dell'accumulatore resta in .A per tutta l'elaborazione e si scrive in
; memoria una sola volta, alla fine.
;
; I cicli medi con operandi casuali comprendono il caricamento dei registri
; e l'rts, ma non la jsr.
;
;   Routine  Senza segno  Con segno
;   8x8            167       214
;   16x8           270       315
;   24x8           336       390
;   32x8           415       466
;   16x16          496       555
;   24x16          663       700
;   32x16          818       864
;   24x24          982      1007
;   32x24         1222      1234
;   32x32         1582      1624
```

2.1.3 m_Mul_Negate

```
; ================================================================================
; MACRO: m_Mul_Negate
; --------------------------------------------------------------------------------
; Descrizione:  Nega sul posto un valore a più byte in pagina zero, cioè lo
;               sostituisce con il suo complemento a 2. Serve per rendere
;               positivi gli operandi e per correggere il segno del prodotto.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .address_ (1 byte, Macro)   : Indirizzo in pagina zero del byte meno
;                                   significativo
;   - .bytes_ (1 byte, Macro)     : Numero di byte del valore (da 1 a 8)
;
; Output:
;   - .address_ (1 .. 8 byte, ZP) : Il valore negato, nelle stesse locazioni
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry),
;                       .address_+0 .. .address_+.bytes_-1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

2.1.4 m_Mul_Stage

```
; ============================================================================
; MACRO: m_Mul_Stage
; ----------------------------------------------------------------------------
; Descrizione:  Elabora un byte del moltiplicatore con 8 iterazioni di
;               shift-and-add. Ogni iterazione somma il moltiplicando
;               all'accumulatore alto se il bit in esame vale 1, poi fa scorrere
;               a destra l'accumulatore e il byte in esame. I bit esaminati
;               escono dal byte del moltiplicatore e al loro posto entrano i bit
;               bassi dell'accumulatore, che formano un byte definitivo del
;               prodotto finale.
;
;               Se il moltiplicando ha un solo byte l'accumulatore alto sta
;               tutto in .A. Altrimenti il byte più alto dell'accumulatore sta
;               in .A e gli altri nei byte di MUL_RES che seguono quelli del
;               moltiplicatore; il byte più alto si scrive in memoria solo alla
;               fine di m_Mul_Core.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .stage_ (1 byte, Macro)        : Indice del byte del moltiplicatore (0 =
;                                      il meno significativo)
;   - .mcand_bytes_ (1 byte, Macro)  : Byte del moltiplicando (da 1 a 4)
;   - .mplier_bytes_ (1 byte, Macro) : Byte del moltiplicatore (da 1 a 4)
;   - MUL_1 (1 .. 4 byte, ZP)        : Moltiplicando
;   - MUL_RES (2 .. 8 byte, ZP)      : Moltiplicatore e accumulatore alto
;   - .A (1 byte, CPU)               : Byte più alto dell'accumulatore
;
; Output:
;   - MUL_RES (2 .. 8 byte, ZP)      : Il byte .stage_ diventa un byte
;                                      definitivo del prodotto
;   - .A (1 byte, CPU)               : Byte più alto dell'accumulatore
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y (se .mcand_bytes_ > 1),
;                       .P (Negative, oVerflow, Zero, Carry),
;                      MUL_RES+0 .. MUL_RES+6
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.5 m_Mul_Core

```
; ================================================================================
; MACRO: m_Mul_Core
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due valori senza segno con l'algoritmo shift-and-add
;               elaborando un byte del moltiplicatore alla volta (m_Mul_Stage).
;               Il moltiplicatore sta nei byte bassi di MUL_RES, che a fine
;               operazione contengono il prodotto di .mcand_bytes_ +
;               .mplier_bytes_ byte.
;
;               Conviene usare come moltiplicando l'operando più lungo, perché
;               il costo cresce con il numero di byte del moltiplicatore.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mcand_bytes_ (1 byte, Macro)  : Byte del moltiplicando (da 1 a 4)
;   - .mplier_bytes_ (1 byte, Macro) : Byte del moltiplicatore (da 1 a 4)
;   - MUL_1 (1 .. 4 byte, ZP)        : Moltiplicando (senza segno)
;   - MUL_2 (1 .. 4 byte, ZP)        : Moltiplicatore (senza segno)
;
; Output:
;   - MUL_RES (2 .. 8 byte, ZP)      : Prodotto della moltiplicazione
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y (se .mcand_bytes_ > 1),
;                       .P (Negative, oVerflow, Zero, Carry),
;                      MUL_RES+0 .. MUL_RES+7
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.2 math_div.asm

2.2.1 Descrizione

Divisione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il quoziente ha la lunghezza del dividendo e il resto quella del divisore, senza perdita di precisione.

```
m_Div_Negate      Nega un valore a più byte in pagina zero
m_Div_Roll        Fa scorrere a sinistra un valore a più byte
m_Div_Core_Byte   Nucleo senza segno con divisore a 8 bit
m_Div_Core_Multi  Nucleo senza segno con divisore a più byte
m_Div_Core_8x8    Nucleo senza segno 8x8 bit
m_Div_Core_16x8   Nucleo senza segno 16x8 bit
m_Div_Core_24x8   Nucleo senza segno 24x8 bit
m_Div_Core_32x8   Nucleo senza segno 32x8 bit
m_Div_Core_16x16  Nucleo senza segno 16x16 bit
m_Div_Core_24x16  Nucleo senza segno 24x16 bit
m_Div_Core_32x16  Nucleo senza segno 32x16 bit
m_Div_Core_24x24  Nucleo senza segno 24x24 bit
m_Div_Core_32x24  Nucleo senza segno 32x24 bit
m_Div_Core_32x32  Nucleo senza segno 32x32 bit
DIV_8X8           Divisione 8x8 bit senza segno
SDIV_8X8          Divisione 8x8 bit con segno
DIV_16X8          Divisione 16x8 bit senza segno
SDIV_16X8         Divisione 16x8 bit con segno
DIV_24X8          Divisione 24x8 bit senza segno
SDIV_24X8         Divisione 24x8 bit con segno
DIV_32X8          Divisione 32x8 bit senza segno
SDIV_32X8         Divisione 32x8 bit con segno
DIV_16X16         Divisione 16x16 bit senza segno
SDIV_16X16        Divisione 16x16 bit con segno
DIV_24X16         Divisione 24x16 bit senza segno
SDIV_24X16        Divisione 24x16 bit con segno
DIV_32X16         Divisione 32x16 bit senza segno
SDIV_32X16        Divisione 32x16 bit con segno
DIV_24X24         Divisione 24x24 bit senza segno
SDIV_24X24        Divisione 24x24 bit con segno
DIV_32X24         Divisione 32x24 bit senza segno
SDIV_32X24        Divisione 32x24 bit con segno
DIV_32X32         Divisione 32x32 bit senza segno
SDIV_32X32        Divisione 32x32 bit con segno
```

2.2.2 Note d'uso

Vedi le pagine seguenti.

2.2.2 Note d'uso

```
; NOTA #01
; --------
; Il dividendo ha m byte e il divisore n byte, con m maggiore o uguale a n.
; Il quoziente ha la lunghezza del dividendo e il resto quella del divisore,
; quindi nessuna informazione va persa. I valori a più byte si scambiano con i
; registri dal byte più basso al più alto: byte 0 in .A, byte 1 in .X e byte 2
; in .Y (cioè la terna .YXA, come nell'ordine di memoria). I byte del
; quoziente oltre il terzo si leggono da DIV_QUOT+3.
;
; Il resto si legge sempre da DIV_REM. Quando il resto ha un solo byte ed è
; rimasto un registro libero, la routine lo restituisce anche in quel registro.
;
;   Routine  Dividendo   Divisore    Quoziente        Resto
;   -------  ----------  ----------  ---------------  ------------
;   8x8      .A          .X          .A               .X e DIV_REM
;   16x8     DIV_1 (ZP)  .A          .A, .X           .Y e DIV_REM
;   24x8     DIV_1 (ZP)  .A          .A, .X, .Y       DIV_REM
;   32x8     DIV_1 (ZP)  .A          .A, .X, .Y (*)   DIV_REM
;   16x16    DIV_1 (ZP)  .A, .X      .A, .X           DIV_REM
;   24x16    DIV_1 (ZP)  .A, .X      .A, .X, .Y       DIV_REM
;   32x16    DIV_1 (ZP)  .A, .X      .A, .X, .Y (*)   DIV_REM
;   24x24    DIV_1 (ZP)  .A, .X, .Y  .A, .X, .Y       DIV_REM
;   32x24    DIV_1 (ZP)  .A, .X, .Y  .A, .X, .Y (*)   DIV_REM
;   32x32    DIV_1 (ZP)  DIV_2 (ZP)  .A, .X, .Y (*)   DIV_REM
;
;   (*) Il byte 3 del quoziente si legge da DIV_QUOT+3.
;
; Per le versioni con segno gli operandi sono in complemento a 2. Il resto ha il
; segno del dividendo e il quoziente si tronca verso zero, come nella divisione
; intera dei linguaggi di alto livello.
```

2.2.2 Note d'uso (continua)

```
; NOTA #02
; --------
; Le routine lavorano in pagina zero sui 12 byte da NUMWRK a NUMWRK+11, cioè
; NUMWRK più i primi due byte di FAC, che sono liberi finché non si usa la
; matematica in virgola mobile del BASIC.
;
;   DIV_1     NUMWRK+0 .. NUMWRK+3     Dividendo (primo operando)
;   DIV_QUOT  NUMWRK+0 .. NUMWRK+3     Quoziente, coincide con DIV_1
;   DIV_2     NUMWRK+4 .. NUMWRK+7     Divisore (secondo operando)
;   DIV_REM   NUMWRK+8 .. NUMWRK+11    Resto
;
; DIV_QUOT coincide con DIV_1: il dividendo viene consumato bit dopo bit e al
; suo posto entrano i bit del quoziente. Per questo il dividendo non si
; conserva. Nelle versioni con segno anche il divisore diventa positivo.
;
; La stessa area è condivisa con ZPL_1 e ZPL_2 (puntatori utente a 24/32 bit)
; e con le routine di moltiplicazione: non si possono usare contemporaneamente.
```

```
; NOTA #03
; --------
; Si elabora un bit del dividendo alla volta, dal più alto. Il resto parziale
; scorre a sinistra e riceve il bit; se è maggiore o uguale al divisore si
; sottrae e il bit di quoziente vale 1, altrimenti vale 0 e il resto non si
; tocca: non esiste nessun ripristino. Il bit di quoziente è il Carry lasciato
; dal confronto ed entra dal basso nel dividendo alla rotazione successiva.
; Dividendo e quoziente condividono gli stessi byte e il quoziente non richiede
; nessun passo a parte.
;
; Con un divisore di un byte il resto sta in .A per tutto il tempo. Con più
; byte sta in DIV_REM e il confronto parte dal byte più alto: si ferma appena
; l'esito è noto, cioè di solito dopo un solo byte.
;
; I cicli medi con operandi casuali comprendono il caricamento dei registri
; e l'rts, ma non la jsr. Il divisore è casuale e non nullo, in parte più
; corto dell'operando.
;
;   Routine  Senza segno  Con segno
;   8x8            200       267
;   16x8           451       528
;   24x8           783       869
;   32x8          1195      1289
;   16x16          686       768
;   24x16         1188      1279
;   32x16         1775      1876
;   24x24         1335      1431
;   32x24         1991      2095
;   32x32         2152      2244
```

2.2.2 Note d'uso (continua)

```
; NOTA #04
; --------
; Divisione per zero: la routine esce con il Carry a 1 e .A uguale a
; ERR_DIVISION_BY_ZERO, senza toccare il quoziente e il resto.
;
; Un dividendo nullo non ha un percorso a parte: il ciclo lo elabora come ogni
; altro valore e dà quoziente e resto uguali a zero.
;
; Overflow nelle versioni con segno: l'unico caso è il dividendo più negativo
; diviso per -1, per esempio -128 / -1 a 8 bit. Il quoziente corretto (+128)
; non entra in 8 bit con il segno: la routine restituisce il quoziente
; troncato ai bit bassi ($80, che letto con il segno vale -128), pone il flag
; di oVerflow a 1 e lascia il Carry a 0. Il bit perso è il bit di segno del
; quoziente vero, che qui vale sempre 0: chi vuole il risultato esatto lo
; ricostruisce con un byte in più a zero. In tutti gli altri casi il flag di
; oVerflow vale 0.
```

2.2.3 m_Div_Negate

```
; ================================================================================
; MACRO: m_Div_Negate
; --------------------------------------------------------------------------------
; Descrizione:  Nega sul posto un valore a più byte in pagina zero, cioè lo
;               sostituisce con il suo complemento a 2. Serve per rendere
;               positivi dividendo e divisore e per correggere il segno di
;               quoziente e resto.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .address_ (1 byte, Macro)   : Indirizzo in pagina zero del byte meno
;                                   significativo
;   - .bytes_ (1 byte, Macro)     : Numero di byte del valore (da 1 a 4)
;
; Output:
;   - .address_ (1 .. 4 byte, ZP) : Il valore negato, nelle stesse locazioni
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry),
;                       .address_+0 .. .address_+.bytes_-1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

2.2.4 m_Div_Roll

```
; ================================================================================
; MACRO: m_Div_Roll
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a più byte in pagina
;               zero. Il Carry entra nel bit meno significativo e il bit più
;               alto del valore esce nel Carry.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .address_ (1 byte, Macro)   : Indirizzo in pagina zero del byte meno
;                                   significativo
;   - .bytes_ (1 byte, Macro)     : Numero di byte del valore (da 1 a 4)
;   - Flag Carry (1 bit, .P)      : 0 = entra uno 0; 1 = entra un 1
;
; Output:
;   - .address_ (1 .. 4 byte, ZP) : Il valore scorso di un bit
;   - Flag Carry (1 bit, .P)      : 0 = esce uno 0; 1 = esce un 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, oVerflow, Zero, Carry),
;                       .address_+0 .. .address_+.bytes_-1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

2.2.5 m_Div_Core_Byte

```
; ================================================================================
; MACRO: m_Div_Core_Byte
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno con un divisore a 8 bit e un dividendo da
;               1 a 4 byte. Per ogni bit del dividendo, dal più alto, il resto
;               parziale scorre a sinistra e riceve il bit; se è maggiore o
;               uguale al divisore si sottrae e il bit di quoziente vale 1. Quel
;               bit entra nel dividendo alla rotazione seguente, quindi il
;               quoziente prende il posto del dividendo.
;
;               Il resto parziale sta in .A per tutta l'elaborazione e si scrive
;               in memoria una sola volta, alla fine. Il divisore deve essere
;               diverso da zero: la macro non lo controlla.
;
; Input:
;   - .dvd_bytes_ (1 byte, Macro) : Byte del dividendo (da 1 a 4)
;   - DIV_1 (1 .. 4 byte, ZP)     : Dividendo (senza segno)
;   - DIV_2+0 (1 byte, ZP)        : Divisore (senza segno, diverso da zero)
;
; Output:
;   - DIV_QUOT (1 .. 4 byte, ZP)  : Quoziente
;   - DIV_REM+0 (1 byte, ZP)      : Resto
;   - .A (1 byte, CPU)            : Resto (uguale a DIV_REM+0)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+.dvd_bytes_-1, DIV_REM+0
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```